Hardware & Software Verification

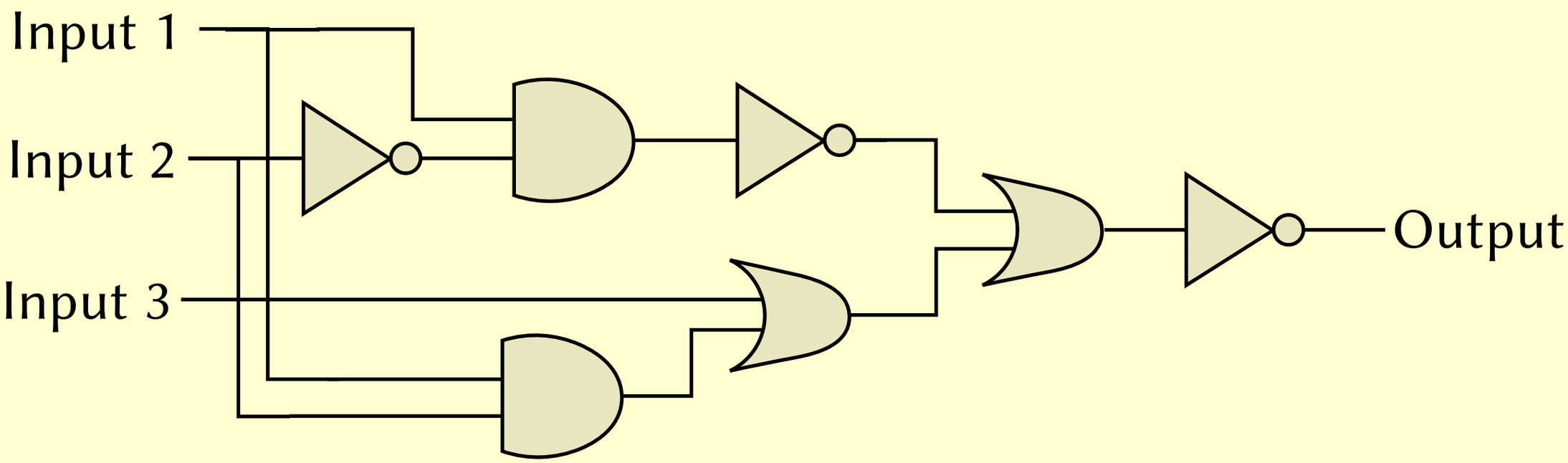
John Wickerson

Lecture 3: More Isabelle
19 October 2026

Lecture Outline

- Proving the correctness of a logic synthesiser.

Representing circuits



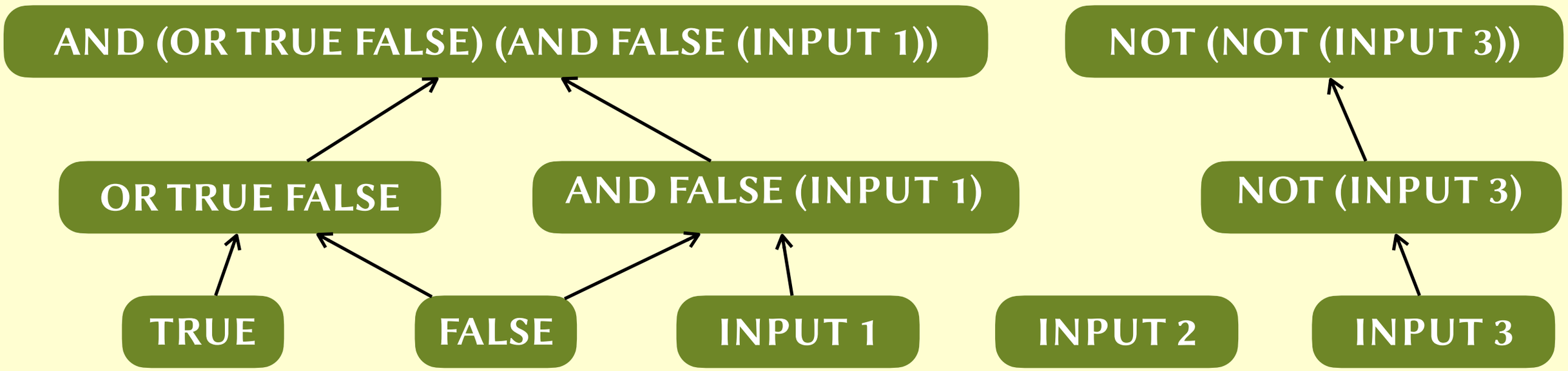

```
NOT
  (OR
    (NOT
      (AND (INPUT 1) (NOT (INPUT 2))))
    (OR
      (INPUT 3)
      (AND (INPUT 1) (INPUT 2))))
```


DEMO

Recursive data structures

```
datatype "circuit" =
  NOT "circuit"
| AND "circuit" "circuit"
| OR "circuit" "circuit"
| TRUE
| FALSE
| INPUT "int"
```

circuit ::= NOT *circuit*
| AND *circuit circuit*
| OR *circuit circuit*
| TRUE
| FALSE
| INPUT *int*



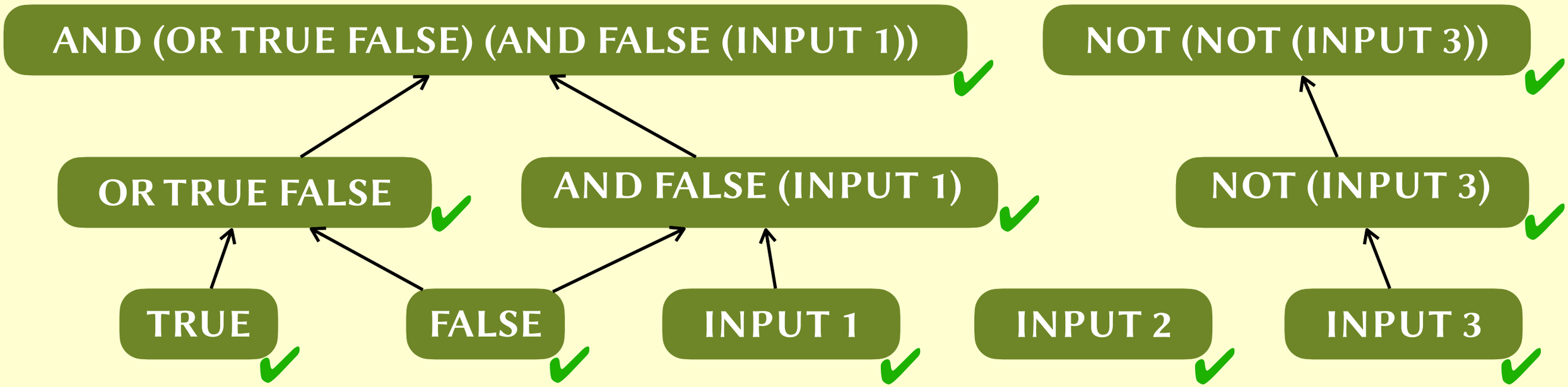

DEMO

Structural induction

- Suppose we want to show that property P holds for all circuits.

- It suffices to show that each constructor preserves P.

1. $\forall c.\ P(c) \Longrightarrow P(\texttt{NOT}\ c)$
2. $\forall c_1, c_2.\ (P(c_1) \wedge P(c_2)) \Longrightarrow P(\texttt{AND}\ c_1\ c_2)$
3. $\forall c_1, c_2.\ (P(c_1) \wedge P(c_2)) \Longrightarrow P(\texttt{OR}\ c_1\ c_2)$
4. $P(\texttt{TRUE})$
5. $P(\texttt{FALSE})$
6. $\forall i.\ P(\texttt{INPUT}\ i)$

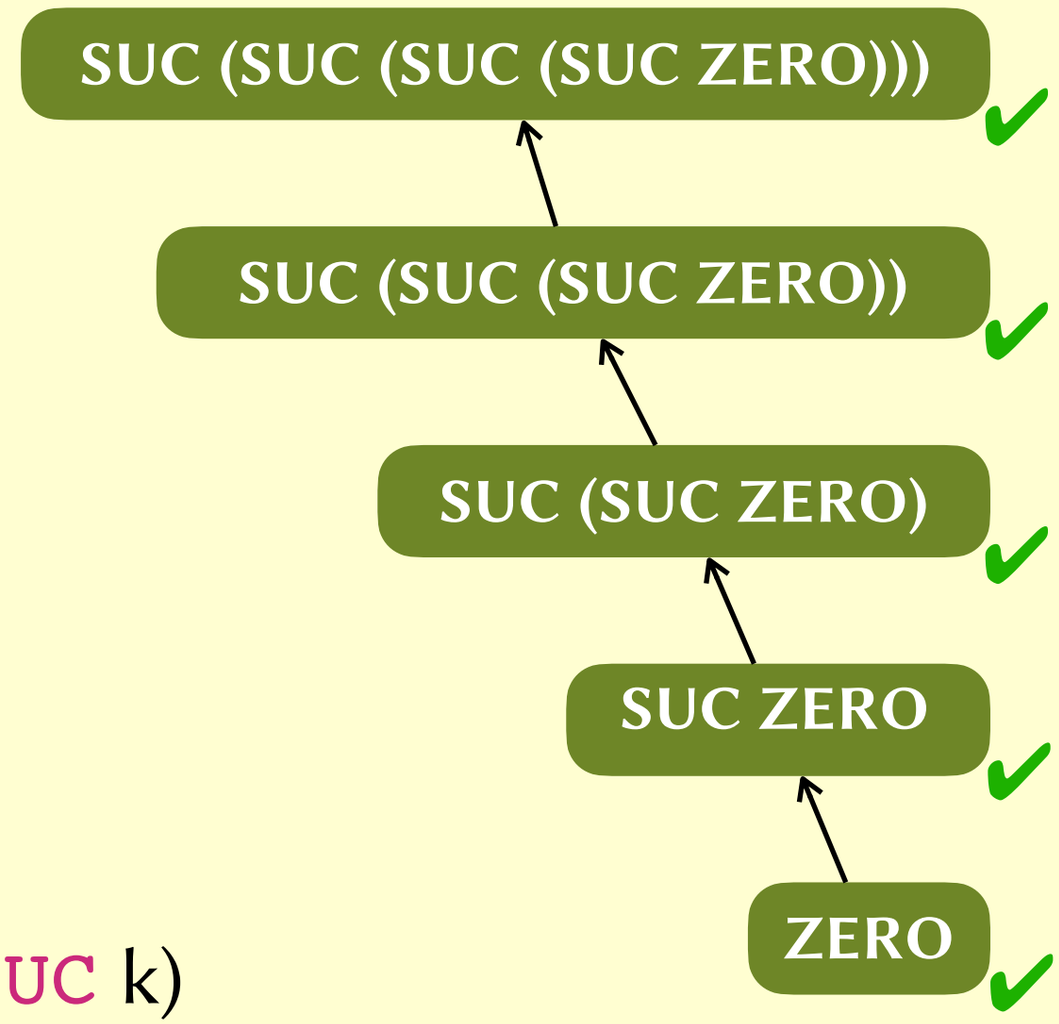


Mathematical induction

```
datatype "nat" =
   ZERO
 | SUC "nat"
```

nat ::= ZERO
| SUC *nat*

1. P(ZERO)
2. ∀k. P(k) ⟹ P(SUC k)



Proof by structural induction

Theorem. simulate (mirror c) ρ = simulate c ρ.

Proof by structural induction

Define P(c) = (∀ρ. simulate (mirror c) ρ = simulate c ρ).

Theorem. P(c) holds for all c.

Proof. We proceed by structural induction on c.

Proof by structural induction

Define P(c) = (∀ρ. simulate (mirror c) ρ = simulate c ρ).

Theorem. P(c) holds for all c.

Proof. We proceed by structural induction on c.

Case NOT. Fix arbitrary c, and assume P(c) as induction hypothesis.

simulate (mirror (NOT c)) ρ

= simulate (NOT (mirror c)) ρ *[by defn of mirror]*

= ¬ simulate (mirror c) ρ *[by defn of simulate]*

= ¬ simulate c ρ *[by induction hypothesis]*

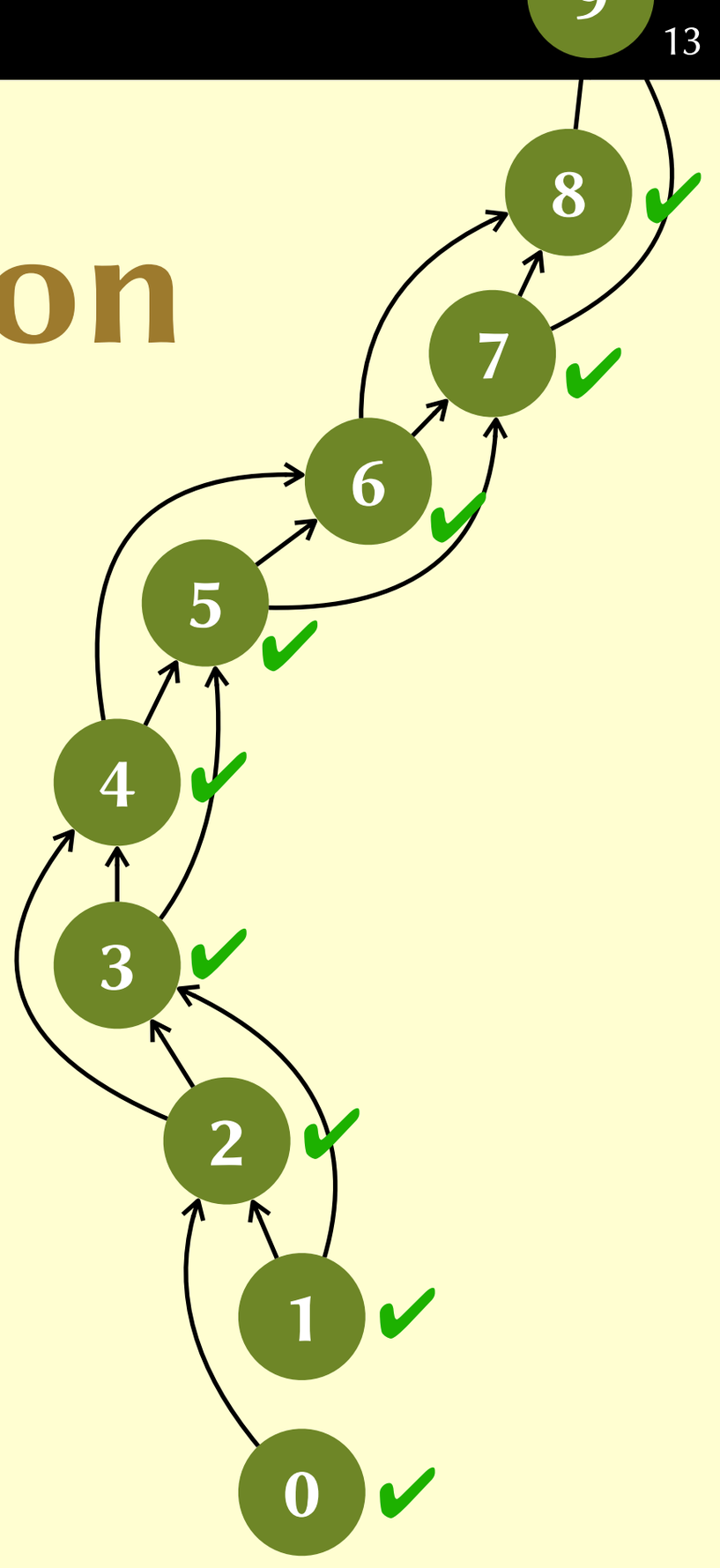
= simulate (NOT c) ρ *[by defn of simulate]*

Thus P(NOT c).


DEMO

Rule induction

```
fun f where
  "f (n+2) = f n + f (n+1)"
| "f 1 = 1"
| "f 0 = 1"
```



1. $\forall n.\ (P(n) \wedge P(n+1)) \Longrightarrow P(n+2)$
2. $P(1)$
3. $P(0)$

Termination

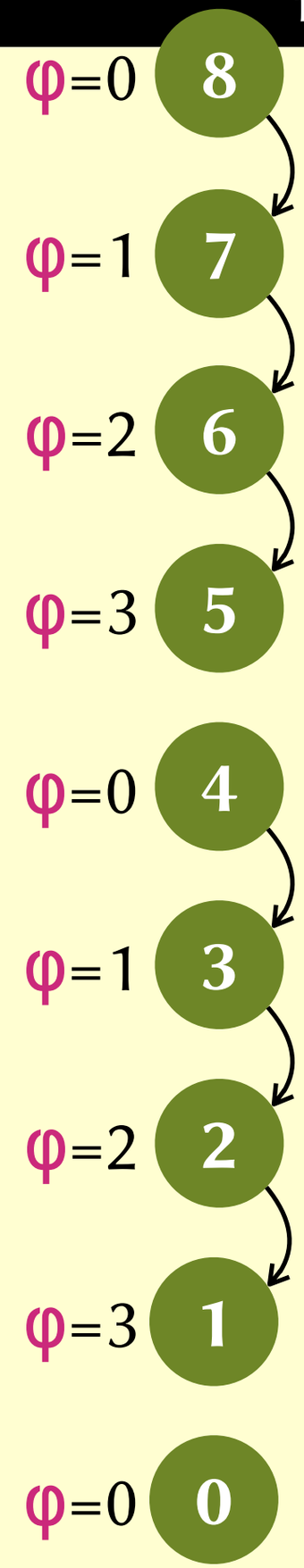
More general version of **fun**.

```
function g :: "nat ⇒ nat" where
  "g n = (if n mod 4 = 0 then n
          else g (n + 1))"
by pat_completeness auto
termination
  apply (relation "measure φ")
  apply auto
  apply presburger
  done
```

Proof that g is defined on all inputs.

Proof that g always terminates, using φ as a termination measure.

```
fun φ :: "nat ⇒ nat" where
  "φ n = 3 - ((n + 3) mod 4)"
```



Summary

- Recursive data structures
- Recursive functions
- Structural induction
- Rule induction
- Termination